

CAT 1 ASSIGNMENT

LINEAR DATA STRUCTURE

SOLUTION A:

Using Doubly Linked Lists

CODE:

dlladt.h

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    char data;
    struct node *next,*prev;
};

void insert(struct node *h,char ele)    ////Insert At End
{
    struct node *ptr=h;
    if (ptr->next==NULL)
    {
        struct node *temp=(struct node*)malloc(sizeof(struct node));
        temp->next=NULL;
        temp->data=ele;
        ptr->next=temp;
        temp->prev=ptr;
    }
    else
    {
        struct node *temp=(struct node*)malloc(sizeof(struct node));
        temp->next=temp->prev=NULL;
        temp->data=ele;
        while (ptr->next!=NULL)
        {
            ptr=ptr->next;
        }
        ptr->next=temp;
        temp->prev=ptr;
    }
}
```

```
void print(struct node *h)
{
    struct node *ptr=h->next;
    while (ptr!=NULL)
    {
        printf("%c ",ptr->data);
        ptr=ptr->next;
    }
}

char forward(struct node *h,struct node *curr[],char ele)
{
    if (curr[0]==NULL)        ///if no cursor has been set,current url will be
null
    {
        struct node *ptr=h->next;
        while(ptr!=NULL)
        {
            if (ptr->data==ele)
            {
                break;
            }
            ptr=ptr->next;
        }
        if (ptr->next!=NULL)    ///ensuring if there exists a forward url
        {
            curr[0]=ptr->next;
            return curr[0]->data;
        }
        else                  ///else return -
            return '-';
    }
    else                      ///cursor has been set already
    {
        if (curr[0]->next==NULL)    ///if cursor next is null
            return '-';

        else                  ///else update cursor and return next url
        {
            curr[0]=curr[0]->next;
            return curr[0]->data;
        }
    }
}
```

```
char backward(struct node *h,struct node *curr[],char ele)
{
    if (curr[0]==NULL)          ////check if the cursor has been set
    {
        struct node *ptr=h->next;
        while (ptr!=NULL)
        {
            if (ptr->data==ele)
                break;
            ptr=ptr->next;
        }
        curr[0]=ptr;
        if (curr[0]->prev!=h)  ////check if a previous url to cursor exists
        {   curr[0]=curr[0]->prev;    ////if yes update cursor and return
the prev url
            return curr[0]->data;
        }
        else                    ////else return -
            return '-';
    }
    else                        ////cursor has been set already
    {
        if (curr[0]->prev==h)  ////if there is no prev url to cursor,return -
            return '-';

        else                    ////if yes,update cursor and return prev url
        {
            curr[0]=curr[0]->prev;
            return curr[0]->data;
        }
    }
}
```

dll.c

```
#include <stdio.h>
#include <stdlib.h>
#include "dlladt.h"
#include <stdbool.h>
void main()
{
    printf("enter the url length :");
    int n;scanf("%d",&n);
    struct node *h=(struct node*)malloc(sizeof(struct node));
    h->next=NULL;h->prev=NULL;
```

```
for (int i=0;i<n;i++)
{
    printf("enter the url one by one :");
    char c;
    scanf(" %c",&c); ////space "% c" to clear the buffer
    insert(h,c);
}

printf("\nenter the current url/window :");
char s;
scanf(" %c",&s);
struct node *curr[1];
curr[0]=NULL;
int ch;
while (true)
{
    printf("\n\n1.FORWARD\n2.BACKWARD\n3.PRINT\n4.BREAK\n");
    printf("Enter your choice : ");
    scanf("%d",&ch);
    if (ch==1)
    {
        printf("Next URL : %c",forward(h,curr,s));
    }
    else if (ch==2)
    {
        printf("Previous URL : %c",backward(h,curr,s));
    }
    else if (ch==3)
    {
        printf("Displaying all the existing urls\n");
        print(h);
    }
    else if (ch==4)
    {
        break;
    }
    else
        printf("\nINVALID CHOICE!\n");
}
```

OUTPUT:

```
C:\Users\User\OneDrive\SEM3\DATA S
enter the url length :7
enter the url one by one :a
enter the url one by one :b
enter the url one by one :c
enter the url one by one :d
enter the url one by one :e
enter the url one by one :f
enter the url one by one :g

enter the current url/window :e
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 1
Next URL : f
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 1
Next URL : g
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 1
Next URL : -
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 2
Previous URL : f
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 2
Previous URL : e
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 2
Previous URL : d
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : |
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 3
Displaying all the existing urls
a b c d e f g
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 4
```

SOLUTION B

CODE:

stackadt.h

```
#include <stdio.h>
#include <stdlib.h>

struct stack
{
    int size;
    int top;
    char arr[100];
    int curr;
};

void create(struct stack *s)
{
    s->size=0;
    s->top=-1;
    s->curr=-1;
}

void push(struct stack *s,int ele)
{
    s->top+=1;
    s->arr[s->top]=ele;
    ++s->size;
}

int isempty(struct stack *s)
{
    if (s->top== -1)
        return 1;
    else
        return 0;
}

int getpos(struct stack *s,int ele)        ////returns the position of a url
{
    int c=0;
    for (int i=0;i<s->size;i++)
    {
        if (ele==s->arr[i])
        {
            return c;
        }
        c++;
    }
}
```

```
    }
}

char forward(struct stack *s,int ele)
{
    if (s->curr==-1)        ////if cursor not set,sets the cursor
    {
        int k=getpos(s,ele);
        s->curr=k;
        if (k+1==s->size)    ////ensure that cursor doesnt grow beyond size
            return '-';

        else
        {
            s->curr=k+1;    ////cursor updated
            return s->arr[s->curr];
        }
    }
    else                    ////cursor already set
    {
        if (s->curr+1==s->size)    ////ensure that cursor doesnt grow beyond
size
            return '-';
        else                    ////update cursor and return fwd url
        {
            s->curr++;
            return s->arr[s->curr];
        }
    }
}

char backward(struct stack *s,int ele)
{
    if (s->curr==-1)        ////if cursor hasnt been set
    {
        int k=getpos(s,ele);
        s->curr=k;        ////update cursor
        if (s->curr-1<0)
            return '-';
        else
        {
            s->curr=k-1;
            return s->arr[s->curr];
        }
    }
    else                    ////if cursor has been already set
    {
        if (s->curr-1<0)
            return '-';
    }
}
```

```
        else
        {
            s->curr--;
            return s->arr[s->curr];
        }
    }
}

char print(struct stack *s)
{
    for (int i=0;i<s->size;i++)
    {
        printf("%c ",s->arr[i]);
    }
}
```

stack.c

```
#include <stdio.h>
#include <stdlib.h>
#include "stackadt.h"
#include <stdbool.h>
void main()
{
    printf("enter the url length :");
    int n;scanf("%d",&n);
    struct stack *h=(struct stack*)malloc(sizeof(struct stack));
    create(h);
    for (int i=0;i<n;i++)
    {
        printf("enter the url one by one :");
        char c;
        scanf(" %c",&c);    ////space "% c" to clear the buffer
        push(h,c);
    }

    printf("\nenter the current url/window :");
    char s;
    scanf(" %c",&s);

    int ch;
    while (true)
    {
        printf("\n\n1.FORWARD\n2.BACKWARD\n3.PRINT\n4.BREAK\n");
        printf("Enter your choice : ");
        scanf("%d",&ch);
```



```
if (ch==1)
{
    printf("Next URL : %c",forward(h,s));

}
else if (ch==2)
{
    printf("Previous URL : %c",backward(h,s));
}
else if (ch==3)
{
    printf("Existing URLS : ");
    print(h);
}
else if (ch==4)
{
    break;
}
else
    printf("\nINVALID CHOICE!\n");
}
```

OUTPUT:

```
enter the url length :7
enter the url one by one :a
enter the url one by one :b
enter the url one by one :c
enter the url one by one :d
enter the url one by one :e
enter the url one by one :f
enter the url one by one :g

enter the current url/window :d
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 2
Previous URL : c
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 2
Previous URL : b
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 2
Previous URL : a
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 2
Previous URL : -
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 1
Next URL : b
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 1
Next URL : c
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 3
Existing URLS : a b c d e f g
```

```
1.FORWARD
2.BACKWARD
3.PRINT
4.BREAK
Enter your choice : 4
```